

第7讲

从 RNN、LSTM 到 Attention

Transformer 为什么会出现？

本讲目标

学完后，你不需要会手写 RNN 或 LSTM，但应该能回答：序列模型为什么需要“记住前文”；普通 RNN 为什么难处理很长依赖；LSTM 改进了什么；早期 Seq2Seq 为什么有信息瓶颈；Attention 为什么是关键转折；以及这些问题为什么最终把研究者推向 Transformer。

先记住一句话

一句话心智模型

Transformer 不是凭空出现的。它是在一条很清楚的技术演化链上长出来的：序列依赖 → RNN 状态传递 → 长期依赖困难 → LSTM 门控记忆 → Seq2Seq 信息瓶颈 → Attention 允许“直接看”相关位置 → Transformer 把 Attention 提升为主角。

关键技术线索：怎样让“远处的信息”更容易参与当前计算？



1. 为什么语言首先是一个“序列问题”？

语言不是一袋互不相关的词。词的顺序会改变含义：“狗咬人”和“人咬狗”包含几乎相同的词，但意义完全不同。

更重要的是，当前词的含义往往依赖很早以前出现的信息。例如：

例子

“小王昨天把新买的电脑放在办公室里。今天早晨，他发现它不见了。”

要理解“它”指什么，模型需要保留前一句中的“电脑”信息。

因此，一个序列模型至少需要解决两个问题：

- 如何把前面已经读过的信息带到后面？
- 当句子很长时，如何避免早期重要信息在传递过程中被冲淡或丢失？

科研视角

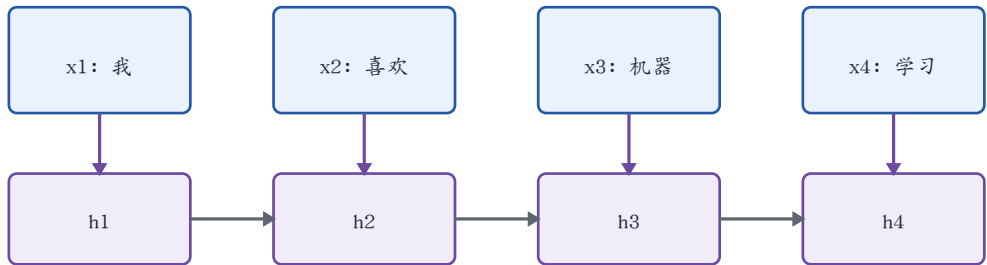
很多架构创新，表面上是换公式，底层往往是在解决“信息怎样流动”的问题。看论文架构图时，先问：信息从哪里来？经过什么路径？到哪里去？

2. RNN 的核心想法：给序列加一个“滚动记忆”

RNN 是 Recurrent Neural

Network，中文常译为循环神经网络。它的核心不是“循环执行代码”，而是：处理第 t 个位置时，把上一步得到的隐藏状态 $h_{(t-1)}$ 继续传进来。

RNN：当前状态 h_t 依赖上一步状态 $h_{(t-1)}$ 和当前输入 x_t



信息沿时间步一步一步传递：天然有顺序依赖

可以把它写成一个非常抽象的式子：

$$h_t = f(W_x x_t + W_h h_{(t-1)} + b)$$

这里你只需要读懂含义：

符号	含义
x_t	当前时间步的输入
$h_{(t-1)}$	前一步保留下来的隐藏状态
h_t	读完当前输入后得到的新状态
W_x, W_h, b	训练得到的参数
f	非线性激活/状态更新函数

通俗比喻

像一个人读文章时手里只有一张不断改写的便签：每读一个词，就把“目前最重要的信息摘要”更新到便签上，然后继续读下一个词。

3. RNN 为什么曾经非常重要？

RNN 第一次把“状态随序列推进”这件事明确地做进神经网络里。它天然适合文本、语音、时间序列等有先后顺序的数据。

与把每个词完全独立处理相比，RNN 能把前文信息带到后文，因此可以完成：

- 语言建模：根据前文预测下一个词；
- 机器翻译：读入源语言序列，再生成目标语言序列；
- 语音识别：根据连续声学帧识别文字；
- 时间序列预测：根据前面时刻的信息预测后续状态。

但它有两个结构性的限制：

限制	直觉
长距离信息传递困难	早期信息必须经过很多时间步，容易被逐渐稀释
顺序计算难并行	第 t 步依赖第 $t-1$ 步，无法像矩阵乘法那样一次把整个序列完全并行处理

重要

这两个限制后来分别推动了两条改进线：LSTM/GRU主要缓解长期依赖；Attention/Transformer则进一步改变了信息访问方式和并行方式。

4. 什么叫“长期依赖”问题？

假设一句话有 100 个 token，第 100 个 token 的正确理解依赖第 3 个 token。普通 RNN 中，第 3 个 token 的信息需要经过 $h_3 \rightarrow h_4 \rightarrow h_5 \rightarrow \dots \rightarrow h_{100}$ 很多次状态更新，才能影响最后的计算。

这时会出现两个层面的困难：

- 表示层面：一张不断被改写的“便签”容量有限，早期细节可能被后续信息覆盖；
- 优化层面：反向传播穿过很多时间步后，梯度可能变得很小或很大，导致很早的状态难以学到正确责任。

梯度多次乘上小于 1 的因子时，会越来越小



$$0.5 \times 0.5 \times 0.5 \times 0.5 = 0.0625 \rightarrow \text{早期时间步收到的学习信号可能非常弱}$$

不要把它简单记成“RNN记性差”

真正的问题是：信息和梯度都要沿着很长的链传递。链越长，稳定地保留并学习远距离关系越困难。

5. 梯度消失与梯度爆炸：为什么训练会变难？

上一讲你已经知道，训练依靠反向传播把梯度从后面传回前面。RNN 在时间上展开以后，本质上就是一个很深的网络。

如果反向传播时不断乘上小于 1 的因子，例如：

$$0.5 \times 0.5 \times 0.5 \times 0.5 \times 0.5 = 0.03125$$

越往前，梯度越小，这叫梯度消失（Vanishing Gradient）。反过来，如果连续乘上大于 1 的因子，梯度可能迅速变得巨大，叫梯度爆炸（Exploding Gradient）。

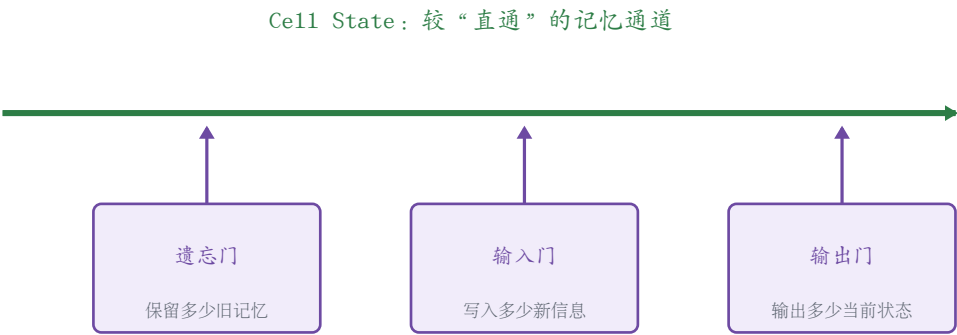
问题	现象	后果
梯度消失	早期时间步梯度接近 0	很早的信息几乎得不到有效学习信号
梯度爆炸	梯度数值急剧增大	训练不稳定，参数更新可能失控

底层原理

这不是“RNN 专属魔咒”，而是深层链式计算中的一般问题。RNN 只是因为同一状态转换在时间轴上重复很多次，所以这个问题特别突出。

6. LSTM：给 RNN 增加“门控记忆”

LSTM 是 Long Short-Term Memory，中文常译为长短期记忆网络。它由 Sepp Hochreiter 与 Jürgen Schmidhuber 在 1997 年提出，目标之一就是缓解普通 RNN 的长期依赖与梯度传播困难。



LSTM 用门控机制控制“忘什么、写什么、输出什么”，缓解普通 RNN 的长期依赖困难

你现在不需要记公式，只要先理解三个门的作用：

门	通俗含义	问题
Forget Gate 遗忘门	旧记忆保留多少	哪些旧信息已经不重要？
Input Gate 输入门	新信息写进去多少	当前输入里哪些内容值得记住？
Output Gate 输出门	当前状态暴露多少	下一步/当前输出需要看到哪些信息？

比喻

普通 RNN 像每读一个词就把整张便签重写一次；LSTM 更像有一个“记忆本”，并配有橡皮擦、写入开关和读取开关。

7. LSTM 解决了一切吗？没有

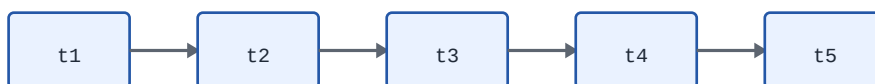
LSTM

确实让序列模型在很多任务上更强，也成为深度学习早期处理文本与语音的重要工具。但它仍然保留了 RNN 的一个基本特征：序列需要按时间步递推。

也就是说：

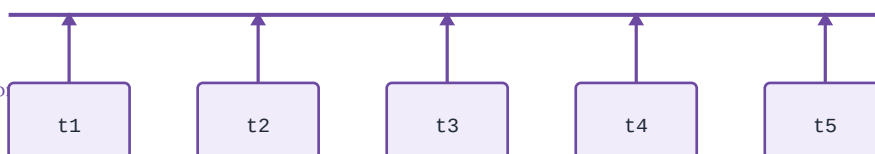
时间步	计算关系
t=1	先算 h_1
t=2	必须拿到 h_1 才能算 h_2
t=3	必须拿到 h_2 才能算 h_3
...	一直如此

RNN



后一步依赖前一步 → 时间维很难完全并行

Self-Attention



同一层里，各位置的 Q/K/V 可用矩阵运算批量计算 → 更适合 GPU 并行

当数据和模型越来越大时，这个顺序依赖会成为硬件效率问题：GPU 很擅长同时做大规模矩阵运算，但 RNN 的时间维计算很难完全并行。

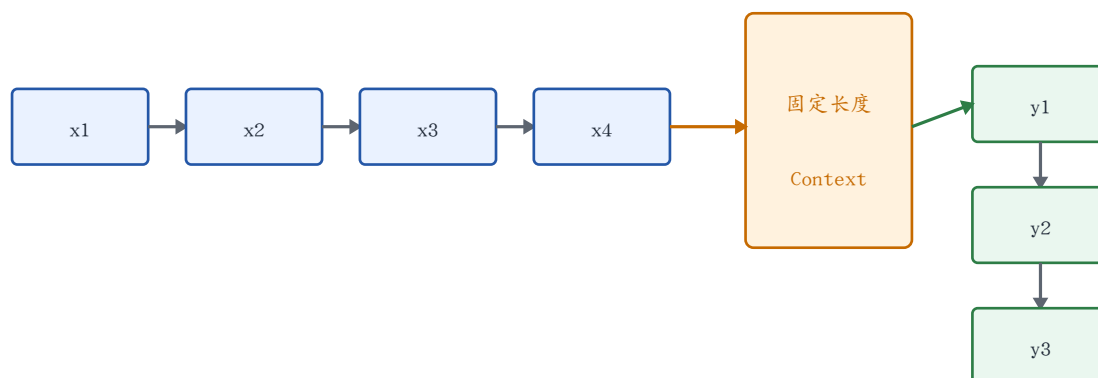
AI Systems 视角

架构优劣不仅是“精度更高不高”。能否并行、能否充分利用 GPU、内存访问模式是否友好，都会决定一个架构能否被真正做大。

8. Seq2Seq：从“一个序列”变成“输入序列 → 输出序列”

机器翻译把问题变成：输入一串词，输出另一串词。早期神经机器翻译常使用 Encoder-Decoder / Seq2Seq 结构。

基本流程：



早期 Seq2Seq：整句信息被压进一个固定长度向量，长句时容易形成“信息瓶颈”

Encoder 逐词读取输入句子，把整句压缩为一个固定长度的上下文向量 Context；Decoder 再根据这个向量逐步生成目标语言。

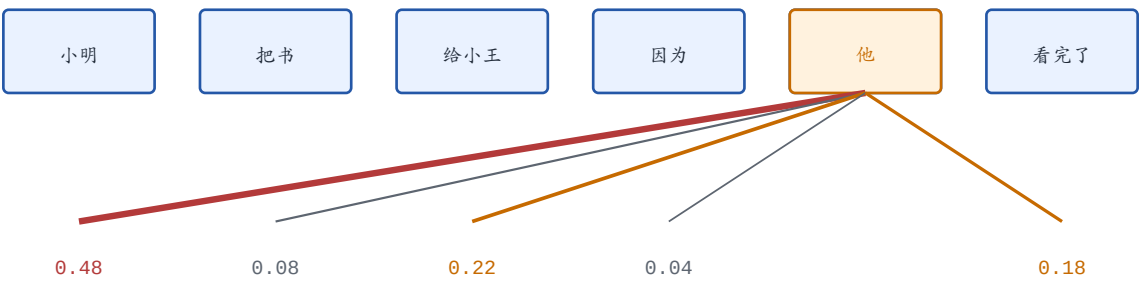
真正的问题

如果输入只有 5 个词，一个向量也许还能压缩得不错；如果输入有 50、100、500 个词，让一个固定长度向量承担“整篇文章摘要”的任务，就会形成明显的信息瓶颈。

9. Attention 的关键转折：别只给 Decoder 一张“总摘要”

Attention 的核心思路非常自然：当 Decoder 生成当前词时，不要只依赖一个固定 Context；让它能够回头查看 Encoder 的所有位置，并动态决定当前最应该关注哪些位置。

Attention 的直觉：处理“他”时，不必只依赖一个压缩状态，而可直接查看句中各位置



权重越大，表示当前计算更重视该位置的信息（这里仅是教学示意，不代表真实模型数值）

于是信息路径发生了根本变化：

早期 Seq2Seq	引入 Attention 后
所有输入信息先压进一个固定 Context	保留每个输入位置的表示
Decoder 只能依赖这份压缩摘要	Decoder 每一步都可以计算对各输入位置的权重
长句容易形成瓶颈	重要位置可被直接访问

一句话

Attention 相当于把“只带一张摘要卡”改成了“可以随时翻整本资料，并给当前最相关的页打高亮”。

11. Attention 为什么对长距离依赖更友好？

在 RNN 中，位置 1 要影响位置 100，信息通常要经过很多次状态传递。Attention 则允许位置 100 直接访问位置 1 的表示。

架构	位置1到位置100的信息路径
RNN/LSTM	$1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow 100$ ，多步传递
Attention	100 可直接对 1 计算注意力并读取其信息

这意味着“远”不再等同于“必须经过很多状态更新”。从信息路径长度看，Attention 大幅缩短了远距离依赖的路径。

但不要过度简化

Attention 并不意味着长上下文问题已经被彻底解决。位置编码、注意力计算成本、有效利用长上下文、Lost in the Middle 等问题仍然存在。这里只是说：信息访问路径发生了重大改善。

12. 从 Attention 到 Self-Attention

最早广为人知的神经机器翻译 Attention，主要用于 Decoder 去“看” Encoder 的各个位置。随后一个很自然的问题出现：同一个句子内部，各个位置之间能不能也直接互相看？这就导向了 Self-Attention（自注意力）：

类型	谁看谁
Encoder-Decoder Attention	Decoder 当前状态 → 看 Encoder 各位置
Self-Attention	同一序列中的每个位置 → 看同一序列中的其他位置

Self-Attention 对语言尤其自然，因为一个词的意义常由整句关系决定。例如“bank”是银行还是河岸，需要看周围语境。

提前预告

下一阶段你会正式看到 Q (Query)、K (Key)、V (Value)：它们就是 Self-Attention 实现“谁要找什么、谁拥有什么标签、真正读取什么内容”的数值机制。

13. Transformer 的关键大胆之处：能不能不要 RNN?

2017 年, Vaswani 等人在论文 Attention Is All You Need 中提出 Transformer。最醒目的思想之一就是：序列建模不再把循环结构作为核心，主要依靠 Attention 加上前馈网络等模块。

这不是说“以前所有东西都没用了”，而是把架构中心从：

传统序列模型中心	Transformer 中心
递归状态 h_t	每层中的 Self-Attention + Feed-Forward
时间步逐步传递	一层内可用矩阵方式处理全部位置
顺序关系内含在递归里	顺序需要额外的位置编码

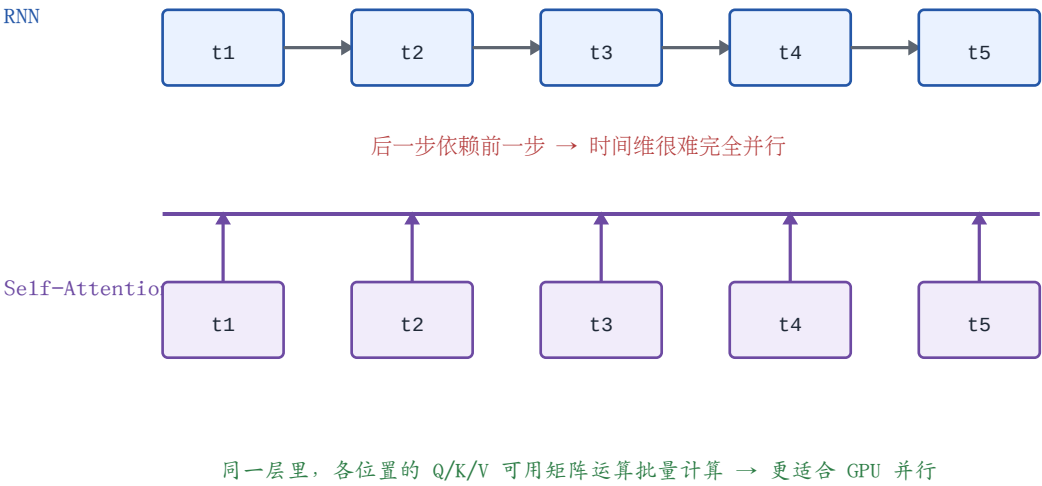
技术史的真正转折

Attention 最初更像“给 Seq2Seq 加一个更好的读取机制”；Transformer 则进一步问：既然 Attention 已经能直接组织序列内部的信息交换，还必须保留循环主干吗？

14. 为什么 Transformer 更适合“大模型时代”？

Transformer

后来成为大语言模型主流架构，并不只因为它“更聪明”，还因为它与现代计算硬件的匹配非常关键。



因素	为什么重要
并行训练	同一层多个 token 的矩阵计算可批量执行，更容易利用 GPU/TPU
远距离交互	一个位置可以直接读取远处位置，不必经过长递归链
可扩展性	随着数据、参数和算力扩大，架构非常适合大规模训练工程
模块化	Attention、FFN、Residual、Norm 等模块可以重复堆叠

关键认识

“能否扩展到更大规模”本身就是算法能力的一部分。一个架构如果理论上很好，但极难并行训练，就很难成为今天数千亿参数模型的基础。

15. 但 Attention 也有代价

Self-Attention 让每个位置都可以和其他位置建立联系。最直接的标准实现中，序列长度为 n 时，需要形成大约 $n \times n$ 的位置关系。

举例：

序列长度 n	两两关系数量 n （示意）
1,000	1,000,000
10,000	100,000,000
100,000	10,000,000,000

因此 Transformer 虽然解决了 RNN 的许多问题，却引出了新的系统问题：长上下文的计算量、显存、KV Cache、带宽以及有效注意力机制。

科研启示

技术进步经常不是“旧问题消失”，而是“把旧瓶颈换成了新瓶颈”。RNN 的关键瓶颈是顺序递归与长期依赖；Transformer 的核心瓶颈之一则转向长序列计算和内存。

16. 一张表看懂 RNN、LSTM、Attention、Transformer

方法	核心机制	主要改进	仍有的关键限制
RNN	隐藏状态递归传递	让神经网络处理序列上下文	长依赖困难；时间维顺序计算
LSTM	门控 + Cell State	更稳定地保留长期信息	仍然是递归序列计算
Attention	动态计算对各位置的权重	可直接读取相关位置，缓解固定 Context 瓶颈	通常仍作为其他架构中的模块
Transformer	Self-Attention + FFN 为主干	去掉循环主干，更强并行、更短信息路径	标准 Attention 对长序列成本高

不要把它们理解成“后一个把前一个彻底淘汰”。在不同任务和资源约束下，RNN/LSTM 仍可能有价值。这里讨论的是：为什么现代通用大语言模型主要走向 Transformer。

17. 从今天的 LLM 反推这段历史

现在把你已经学过的 Token 和 Embedding 接回来:



第7讲解释的其实就是方括号里的历史选择：序列内部的信息到底怎样交换。

阶段	主要信息交换方式
RNN	沿时间轴用隐藏状态传递
LSTM	仍沿时间轴，但用门控和 Cell State 改善记忆
Attention	当前计算可直接从多个位置读取信息
Transformer	把 Self-Attention 变成层内主要的信息交换机制

与后续课程的接口

下一步你会真正拆 Transformer：先学 Attention，再学 Q/K/V、多头注意力、位置编码、Residual、LayerNorm、FFN，最后重新拼成完整 Transformer Block。

18. 和 Agent Systems 有什么关系？

你研究 Agent Systems 时虽然通常不会自己重新发明 Transformer，但底层架构会不断影响 Agent 的行为与系统成本。

- 长上下文 Agent: Agent 把工具结果、历史对话、Memory 都塞入上下文时，Attention 的长序列成本会快速上升；
- 多 Agent 通信: Agent 之间交换的消息最终变成 token，进入 Transformer 的上下文并参与 Attention；
- 上下文工程: 把什么信息放在前面/后面、是否压缩、是否检索，本质上是在管理模型能够关注的信息；
- 系统性能: Prefill、KV Cache、长上下文延迟等都与 Transformer 的推理机制相关；
- 可靠性: 即使信息已经放进上下文，也不代表模型一定会有效利用它，Attention 权重与最终行为之间也不能简单等同。

科研切入点意识

以后看到“Agent 忘了早期规则”“长任务后偏离目标”“多 Agent 消息越来越长”，不要只从 Prompt 层思考。其背后同时存在模型的上下文利用能力、注意力结构、系统缓存与信息管理工作。

19. 常见误区

误区1：LSTM 就是“永久记忆”

不是。它只是通过门控改善长期依赖，仍会丢信息，也受模型容量和训练限制。

误区2：Attention 等于“模型真的把注意力放在某件事上”

注意力权重是数值计算机制。它和人类心理学意义上的“注意”不能直接画等号。

误区3：Transformer 完全不关心顺序

错误。它去掉了 RNN 的递归顺序机制，所以反而必须通过位置编码等方式显式提供位置信息。

误区4：Attention 解决了所有长上下文问题

没有。标准注意力的 n^2 成本、长距离有效利用、位置编码和缓存等仍然是重要问题。

误区5：Transformer 强只是因为算法更好

不完整。它和 GPU/TPU 的大规模并行计算非常匹配，这种“系统可扩展性”也是它成功的重要原因。

误区6：RNN/LSTM 已经没用了

也不对。在流式、低资源、特定时序任务中，它们仍可能更合适。

20. 小实验：用一个玩具 Attention 看“加权读取”

材料包里的 `04_toy_attention_demo.py` 不实现完整 Transformer，只做 Attention 最核心的“相关性 $\rightarrow$ Softmax $\rightarrow$ 加权求和”。

程序给三个“记忆向量”以及一个当前查询向量，先计算相似度，然后把相似度变成权重，最后根据权重把信息加权汇总。

你会看到类似：

```
scores -> [2.0, 0.5, -0.2] softmax -> [0.7506, 0.1675, 0.0819] context -> 0.7506*v1 + 0.1675*v2 + 0.0819*v3
```

你现在不需要推导 Softmax，只要观察：分数越高的位置，最终读取权重越大。

实验目的

把“Attention 会看不同位置”从一句话变成一个可以看到数值的计算过程。下一讲以后再逐步增加 Q/K/V 和矩阵形式。

21. 本讲总结：只记住这条演化链

关键技术线索：怎样让“远处的信息”更容易参与当前计算？



请把下面 7 句话真正理解，而不是背下来：

- 语言是序列，当前含义依赖前文。
- RNN 用隐藏状态把前文信息一步步带到后面。
- 很长的递归链会带来长期依赖、梯度传播和并行计算问题。
- LSTM 用门控与 Cell State 改善长期信息保留，但仍然顺序递归。
- 早期 Seq2Seq 把整句压成固定 Context，长句容易形成信息瓶颈。
- Attention 允许当前计算直接读取多个位置，并动态决定权重。
- Transformer 进一步把 Self-Attention 提升为主干，去掉循环主干，为大规模并行训练奠定基础。

下一讲

第8讲将正式进入 Transformer 的核心机制：Attention 到底是什么？

我们会从“相关性”开始，逐步进入 Query、Key、Value，为后面的 Self-Attention 公式做准备。

22. 术语表

术语	英文	零基础解释
循环神经网络	RNN / Recurrent Neural Network	让隐藏状态沿序列逐步传递的网络
隐藏状态	Hidden State	当前时间步携带的内部表示/摘要
长期依赖	Long-term Dependency	当前计算需要很久以前的信息
梯度消失	Vanishing Gradient	反向传播到很早位置时梯度变得很小
梯度爆炸	Exploding Gradient	反向传播中梯度迅速变得巨大
长短期记忆网络	LSTM	用门控和 Cell State 改善长期依赖的 RNN 变体
序列到序列	Seq2Seq	输入一个序列，输出另一个序列
编码器	Encoder	把输入序列转换为内部表示
解码器	Decoder	根据内部表示生成输出序列
注意力	Attention	按相关性为多个位置分配权重并读取信息
自注意力	Self-Attention	同一序列内部的位置彼此计算注意力
Transformer	Transformer	以 Attention/FFN 等模块为核心的现代序列架构
并行计算	Parallel Computation	多个位置/矩阵运算可同时执行

23. 练习题（先自己做）

- 1. 为什么语言模型不能把每个 token 完全独立处理？
- 2. RNN 的隐藏状态 h_t 起什么作用？
- 3. 为什么位置 1 的信息要影响位置 100 时，普通 RNN 会比较困难？
- 4. 梯度消失是什么？它为什么会影响长期依赖学习？
- 5. LSTM 的遗忘门、输入门、输出门分别解决什么直觉问题？
- 6. LSTM 已经缓解长期依赖，为什么后来还需要 Attention？
- 7. 早期 Seq2Seq 的固定 Context 为什么可能成为瓶颈？
- 8. 用一句话解释 Attention 的核心思想。
- 9. Attention 为什么能缩短远距离信息的访问路径？
- 10. Self-Attention 和 Encoder-Decoder Attention 的“谁看谁”有什么区别？
- 11. Transformer 为什么更容易利用 GPU 并行？
- 12. Transformer 是否“完全不需要顺序信息”？为什么？
- 13. 标准 Self-Attention 在很长序列上会带来什么新问题？
- 14. 请画出你自己的技术演化链：RNN $\rightarrow$ LSTM $\rightarrow$ Attention $\rightarrow$ Transformer，并为每一步写一个“它主要解决了什么”。
- 15. 结合 Agent Systems：为什么一个 Agent 的上下文越来越长会成为系统问题？

24. 练习题答案与自检

第1题

因为语言含义依赖上下文和词序；同一个 token 在不同上下文可能表示不同含义。

第2题

h_t 是读到当前位置后的内部状态，它把前面已经处理的信息带到下一时间步。

第3题

信息需要经过许多次 h 的递推，表示可能被覆盖；反向传播也需要穿过很长计算链。

第4题

梯度在链式传播中越来越小，早期位置几乎收不到学习信号，因而难以学到远距离依赖。

第5题

遗忘门决定旧信息保留多少；输入门决定新信息写多少；输出门决定当前状态输出多少。

第6题

LSTM 仍按时间步顺序递推，难完全并行；长序列的信息访问仍需要经过递归链。

第7题

整句所有信息被压缩成一个固定长度向量，句子越长越容易丢失细节。

第8题

当前计算根据相关性给多个位置分配权重，再按权重读取和汇总这些位置的信息。

第9题

目标位置可以直接访问远处位置，不必经由每一个中间时间步逐级传递。

第10题

Encoder-Decoder Attention 是 Decoder 看 Encoder；Self-Attention 是同一序列内部各位置彼此看。

第11题

一层内各 token 的矩阵计算可以批量执行，不必等待前一个时间步先完成。

第12题

不是。Transformer 去掉了递归，所以需要位置编码等机制把顺序/位置提供给模型。

第13题

标准实现的注意力系数随 n 增长，带来计算、显存和带宽压力。

第14题

RNN: 引入序列状态; LSTM: 改善长期记忆/梯度; Attention: 动态直接读取相关位置; Transformer: 以 Self-Attention 为主干并提高并行能力。

第15题

更长上下文意味着更多 token 参与 Attention、Prefill 和缓存，增加延迟、算力和内存，同时模型也未必能有效利用全部信息。

25. 推荐原始论文（知道它们是谁即可）

现阶段不要求你精读这些论文。先把名字和历史位置建立起来，后面学完 Transformer 再回头读，会容易很多。

年份	代表工作	你现在只需知道
1997	Long Short-Term Memory — Hochreiter & Schmidhuber	LSTM 的经典来源
2014	Neural Machine Translation by Jointly Learning to Align and Translate — Bahdanau et al.	Attention 在神经机器翻译中的代表性早期工作
2014	Sequence to Sequence Learning with Neural Networks — Sutskever et al.	经典 Seq2Seq 工作
2017	Attention Is All You Need — Vaswani et al.	Transformer 的奠基论文

学习策略

这门课的顺序是“先有心智模型，再看公式，再读论文”。现在直接啃原论文，很多符号只会形成负担。等学完 Attention、Multi-Head、位置编码和 Transformer Block，再精读 2017 论文。